Solution 1

Solution 1 (a)

Online Algorithm Implementation: Mean

```
# load libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# quick check data-1.txt has 5000 pts
# cat data-1.txt | wc -l -> 5000

# load data-1.txt
data = np.loadtxt('data-1.txt')

def online_algorithm_mean(data):
    mean_i = []
    dt = []
    current_mean = 0
    for t,x_t in enumerate(data):
        dt.append(t+1)
        current_mean = current_mean + ((x_t-current_mean)/(t+1))
        mean_i.append(current_mean)
    print(f"final mean: {mean_i[-1]}")
    return dt, mean_i

def plot_mean_dt(dt,mean_i,true_mean):
    fig,ax = plt.subplots(nrows=1,ncols=1,figsize=(8,10))
    sns.lineplot(x=dt,y=mean_i,markers='o',ax=ax,color='blue',label='OA')
    sns.lineplot(x=dt,y=[true_mean for i in
        dt],markers='--',ax=ax,color='red',label='True Mean')
    ax.set_xlabel('dt')
    ax.set_ylabel('mean')
    ax.set_ylim(min(mean_i)-1e5,true_mean+1e5)
    print(min(mean_i))
    title = f"Online Algorithim(OA) for Mean\n True Mean: {true_mean:.3f} \n OA Mean:
        {mean_i[-1]:.3f}"
    ax.set_title(title)
    fig.savefig('hw3_q1.png')

# calc true mean and print it
true_mean = sum(data)/len(data)
print(true_mean)

dt, mean_i = online_algorithm_mean(data)
plot_mean_dt(dt,mean_i,true_mean)
```

Comments

The true mean is $2,991,834.673$ and is identical to the final mean calculated via `online_algorithm_mean(data)` function.

Solution 1

Solution 1 (b)

Plot

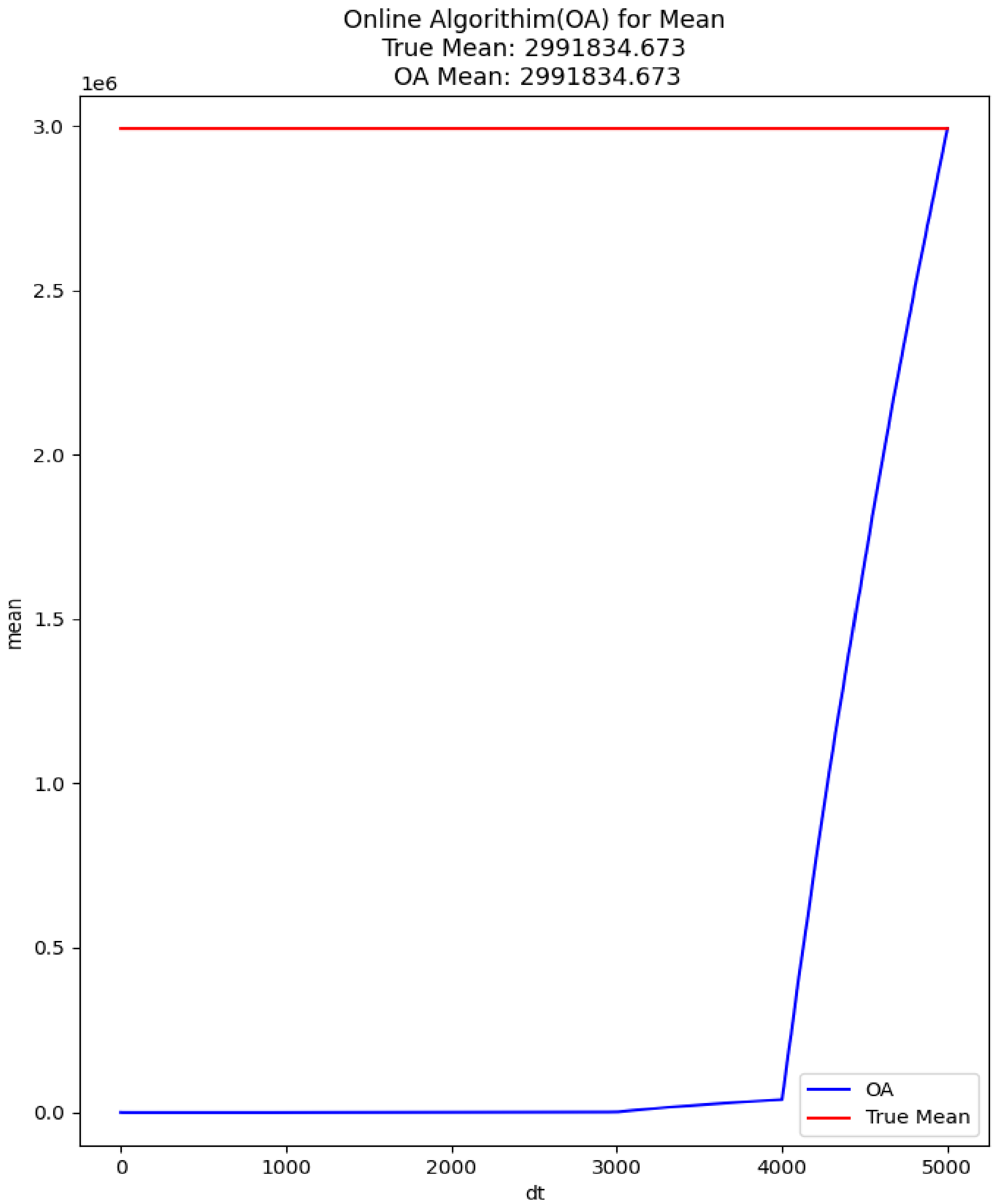

Figure 1: Figure shows all 5000 intermediate values of mean

Solution 2

Solution 2 (a)

Pseudocode

Algorithm 1 Online Variance Calculation

Initialize:
- $t \leftarrow 0$ ▷ Count of data points
- $\mu \leftarrow 0.0$ ▷ Running mean of X
- $M_2 \leftarrow 0.0$ ▷ Running mean of X^2
- $v \leftarrow 0.0$ ▷ Running variance

For each new data point x_t:
- $t \leftarrow t + 1$
- $\mu \leftarrow \mu + (x_t - \mu)/t$ ▷ Update mean of X
- $M_2 \leftarrow M_2 + (x_t^2 - M_2)/t$ ▷ Update mean of X^2
- $v \leftarrow M_2 - \mu^2$ ▷ Calculate current variance

Solution 2

Solution 2 (b)

Online Algorithm Implementation: Variance

```python
# load libraries
import numpy as np

# load data-1.txt
data = np.loadtxt('data-1.txt')

def online_algorithm_variance(data):
    mean_x = 0.0
    mean_x_squared = 0.0
    variances = []

    for t,x_t in enumerate(data):
        # Update the mean of x
        mean_x = mean_x + ((x_t-mean_x)/(t+1))
        # Update the mean of x-squared
        mean_x_squared = mean_x_squared + ((x_t**2-mean_x_squared)/(t+1))

        variances.append(mean_x_squared - mean_x**2)

    print(f"final variance: {variances[-1]:.3f}")
    return variances

# calc true variance and print
true_variance = np.var(data)
print(f"true variance: {true_variance:.3f}")

# calc online_algorithm_variance
variances = online_algorithm_variance(data)
```

Comments

The true variance using numpy: $36,618,966,207,041.938$ while the variance calculated via `online_algorithm_variance(data)` was: $36,618,966,207,041.961$. This discrepancy between the true and *online variance calculation* arises because the online formula is sensitive to floating point precision errors, especially when subtracting two large, similar numbers.

Solution 3

Solution 3 (a)

True Median

```python
# load libraries
import numpy as np

# load data-1.txt
data = np.loadtxt('data-1.txt')

# calc true median
true_median = np.median(data)
print(f"true median: {true_median:.3f}")
```

The true median using numpy: $1,526$

Solution 3

Solution 3 (b)/(c)

Random Sample with Replacement Algorithm Implementation: Median

```
# load libraries
import numpy as np
import random

# load data-1.txt
data = np.loadtxt('data-1.txt')

def estimate_online_median(data, k, repetitions):

    print(f"--- Starting estimations for k={k} ---")
    for rep in range(repetitions):
        # initialize the sample array 's' with the first data point.
        # from s1=s2=...=sk=x1
        s = np.full(k, data[0])

        # get x_t from t=2,3,...,t
        for i in range(1, n):
            t = i + 1
            x_t = data[i]

            # for j=1,2,...,k then sj = xt with probability 1/t.
            for j in range(k):
                if random.random() < (1.0 / t):
                    s[j] = x_t

        # calculate the median of the final sample for each repetition
        estimated_median = np.median(s)
        print(f"repetition {rep + 1:2d}: estimated median = {estimated_median}")
    print("-" * (35 + len(str(k))))


# part (a): calc and print the true median
true_median = np.median(data)
print("--- true median calculation ---")
print(f"true median: {true_median}\n")

# part (b): simulation for k=100
estimate_online_median(data, k=100, repetitions=10)

# part (c): simulation for k=500
estimate_online_median(data, k=500, repetitions=10)
```

Solution 3

Solution 3 (b)/(c)

Random Sample with Replacement Algorithm Results: Median

(a) Estimated Medians for $k = 100$

Repetition	Estimated Median
1	1750.0
2	1094.0
3	1236.0
4	1166.0
5	1552.5
6	1338.5
7	1583.5
8	1635.0
9	1805.5
10	1376.0

(b) Estimated Medians for $k = 500$

Repetition	Estimated Median
1	1422.0
2	1498.5
3	1404.5
4	1328.5
5	1592.5
6	1452.5
7	1520.5
8	1444.5
9	1541.5
10	1567.0

Table 1: Comparison of estimated medians for different sample sizes. *Note: as k grows sufficiently large the calculation for the median of a sample using the random sample with replacement should converge to the true median.*

Solution 4

An ϵ-heavy hitter is an element in a data stream of length m whose frequency is greater than or equal to ϵm. We need to find the maximum number of such elements when $\epsilon = 0.05$.
Let k be the number of distinct ϵ-heavy hitters. Let the frequency of the i-th heavy hitter be denoted by f_i.
By definition, for each of the k heavy hitters:

$$f_i \geq \epsilon m$$

The sum of frequencies of these distinct items cannot exceed the total stream length m:

$$\sum_{i=1}^{k} f_i \leq m$$

Substitute the minimum frequency for each heavy hitter into the sum:

$$\sum_{i=1}^{k} (\epsilon m) \leq \sum_{i=1}^{k} f_i \leq m$$
$$k \cdot \epsilon \cdot m \leq m$$

Since $m > 0$, we can divide both sides by m:

$$k \cdot \epsilon \leq 1$$

Solving for k:

$$k \leq \frac{1}{\epsilon}$$
$$k \leq \frac{1}{0.05}$$
$$k \leq 20$$

$\therefore$ $k \leq 20$ demonstrates that for $\epsilon = 0.05$, there can be at most 20 ϵ-heavy hitters in the stream.

Solution 5

Solution 5 (a)

Optimal Clusters

The k-means objective is to find a set of k centers $M = \{\mu_1, \ldots, \mu_k\}$ that minimizes the sum of squared Euclidean distances from each data point to its nearest center. By inspection, the optimal partitioning separates the negative numbers, the positive numbers, and the origin.

Hence, the optimal clusters are $C_1^* = \{-10, -8\}$, $C_2^* = \{0\}$, and $C_3^* = \{8, 10\}$.

Optimal Centers

The optimal centers μ_j^* are calculated as the mean of the points in each optimal cluster C_j^*:

$$\begin{aligned}
\mu_1^* =& \frac{-10 + (-8)}{2} = -9 \\
\mu_2^* =& \frac{0}{1} = 0 \\
\mu_3^* =& \frac{8 + 10}{2} = 9
\end{aligned}$$

Hence, the optimal centers are:

$$\mu_1^* = -9 \quad \mu_2^* = 0 \quad \mu_3^* = 9$$

Optimal Cost

The globally optimal cost, J^*, is the sum of squared distances for this configuration:

$$\begin{aligned}
J^* &= \sum_{x \in C_1^*} (x - \mu_1^*)^2 + \sum_{x \in C_2^*} (x - \mu_2^*)^2 + \sum_{x \in C_3^*} (x - \mu_3^*)^2 \\
&= \big((-10 - (-9))^2 + (-8 - (-9))^2\big) + \big((0 - 0)^2\big) + \big((8 - 9)^2 + (10 - 9)^2\big) \\
&= \big((-1)^2 + 1^2\big) + 0 + \big((-1)^2 + 1^2\big) \\
&= (1 + 1) + 0 + (1 + 1) \\
&= 4
\end{aligned}$$

$\therefore$ The optimal centers are $\{-9, 0, 9\}$ with a minimum cost of $J^* = 4$.

Solution 5

Solution 5 (b)

Let

$$\mu_1 = -10 \quad \mu_2 = -8 \quad \mu_3 = 0 \quad \text{and} \quad k = 3$$

Assignment

Assign each point in X to the closest mean.

$$\begin{aligned}
x_1 &= -10 \to \mu_1 \text{ is closest } \to C_1 = \{-10\} \\
x_2 &= -8 \to \mu_2 \text{ is closest } \to C_2 = \{-8\} \\
x_3 &= 0 \to \mu_3 \text{ is closest } \to C_3 = \{0\} \\
x_4 &= 8 \to \mu_3 \text{ is closest } \to C_3 = \{0, 8\} \\
x_5 &= 0 \to \mu_3 \text{ is closest } \to C_3 = \{0, 8, 10\}
\end{aligned}$$

Hence, the resulting clusters are: $C_1 = \{-10\}$ $\quad C_2 = \{-8\}$ $\quad C_3 = \{0, 8, 10\}$

Update Centers

Recalculate centers as means of the new clusters:

$$\mu_1^{(1)} = \frac{-10}{1} = -10 \quad \mu_2^{(1)} = \frac{-8}{1} = -8 \quad \mu_3^{(1)} = \frac{0+8+10}{3} = 6$$

Hence, the new centers are: $M^{(1)} = \{-10, -8, 6\}$

Second Assignment

Assign each point in X to the closest mean.

$$\begin{aligned}
x_1 &= -10 \to \mu_1 \text{ is closest } \to C_1 = \{-10\} \\
x_2 &= -8 \to \mu_2 \text{ is closest } \to C_2 = \{-8\} \\
x_3 &= 0 \to \mu_3 \text{ is closest } \to C_3 = \{0\} \\
x_4 &= 8 \to \mu_3 \text{ is closest } \to C_3 = \{0, 8\} \\
x_5 &= 0 \to \mu_3 \text{ is closest } \to C_3 = \{0, 8, 10\}
\end{aligned}$$

Final Centers

Since the cluster assignments did not change, the centers will not change either.
Hence, the final centers are: $M^{(F)} = \{-10, -8, 6\}$

Final Cost

$$\begin{aligned}
J^F &= \left((-10 - (-10))^2\right) + \left((-8 - (-8))^2\right) + \left((0-6)^2 + (8-6)^2 + (10-6)^2\right) \\
&= 0 + 0 + \left((-6)^2 + 2^2 + 4^2\right) \\
&= 36 + 4 + 16 \\
&= 56
\end{aligned}$$

$\therefore$ The algorithm terminates at the centers $\{-10, -8, 6\}$ with a cost of $J^F = 56$.

Solution 6

Solution 6 (a)

something something something

```python
import numpy as np
import matplotlib.pyplot as plt
"""
code different
"""
data = np.loadtxt
```

Solution 6

Solution 6 (b)

something something something

```
import numpy as np
import matplotlib.pyplot as plt
"""
code different
"""
data = np.loadtxt
```

Solution 7

Solution 7 (a)

something something something

Solution 7

Solution 7 (b)

something something something
